

Facultad de Ciencias, UNAM

Desarrollo Web Backend

Validar token en API Customer/Product

Introducción

Este documento muestra la implementación que se debe agregar a una API para validar la autenticidad de un token.

Contenido

1. Agregar dependencias Jwt
2. Interceptar y guardar el token
3. Restringir acceso a endpoints
4. Probar login

Implementación

Parte 1 - Agregar dependencias Jwt

1. Agregamos las siguientes dependencias que van a ser necesarias para la validación del token:
 - Spring Security
 - Jwt API
 - Jwt Impl
 - Jwt Jackson

```
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-api -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-impl -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-jackson -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Parte 2 - Interceptar y guardar el token

2. En el paquete *config* creamos el paquete *jwt*.
3. Dentro del paquete creamos dos clases Java llamadas *JwtAuthFilter* y *JwtUtil*.

JwtUtil es la clase utilitaria que lee y valida el contenido del JWT. Se encarga de parsear el token con la clave secreta y extraer datos del payload como username, id y roles.

https://github.com/ivansaavedrapm/customer-service/blob/develop/customer-service/src/main/java/com/customer_service/config/jwt/JwtUtil.java

JwtAuthFilter es el filtro de Spring Security que intercepta cada petición HTTP. Si encuentra un token Bearer, usa *JwtUtil* para leerlo y luego guarda la autenticación del usuario en memoria durante esa request, para que Spring pueda autorizar el acceso a los endpoints.

https://github.com/ivansaavedrapm/customer-service/blob/develop/customer-service/src/main/java/com/customer_service/config/jwt/JwtAuthFilter.java

Parte 3 - Restringir acceso a endpoints

4. Una vez validado y guardado el token, lo siguiente es definir a qué endpoints puede acceder cada usuario. Por ejemplo, un usuario *Admin* podría acceder a todos los endpoints, mientras que un usuario *Customer* solo a algunos. Para hacer esto, necesitamos las clases *SecurityConfig* y *CorsConfig*:

SecurityConfig es la clase que configura la seguridad de la API con Spring Security. Ahí se define qué rutas son públicas, cuáles requieren autenticación o permisos específicos, y también se registra el filtro *JwtAuthFilter* para que el token se valide en cada petición.

https://github.com/ivansaavedrapm/customer-service/blob/develop/customer-service/src/main/java/com/customer_service/config/security/SecurityConfig.java

CorsConfig es la clase que configura CORS para permitir solicitudes desde el frontend u otros clientes. Ahí se establecen los orígenes, métodos HTTP y headers permitidos, incluyendo *Authorization*, para que el navegador pueda enviar el token JWT sin bloqueos por política de origen cruzado.

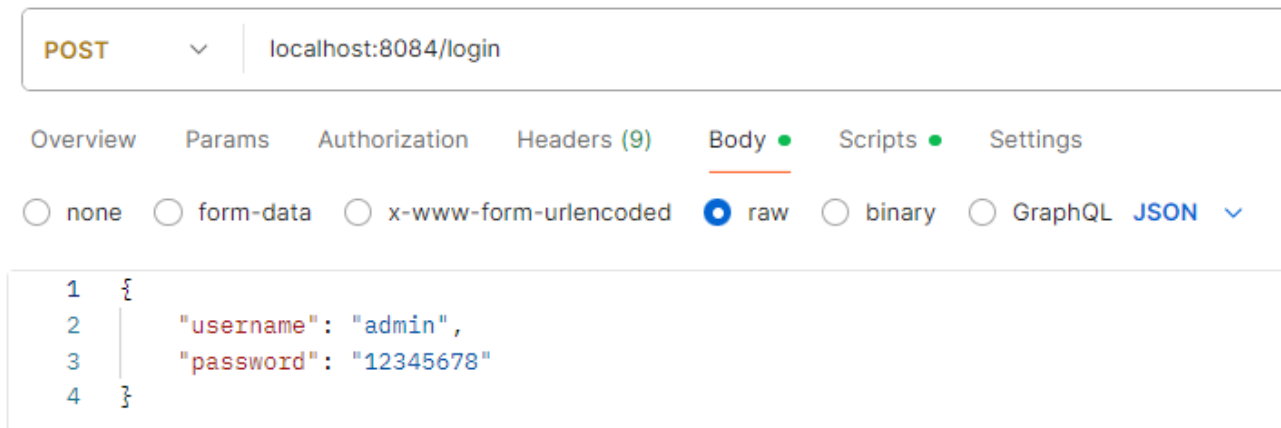
https://github.com/ivansaavedrapm/customer-service/blob/develop/customer-service/src/main/java/com/customer_service/config/security/CorsConfig.java

Parte 4 - Probar login

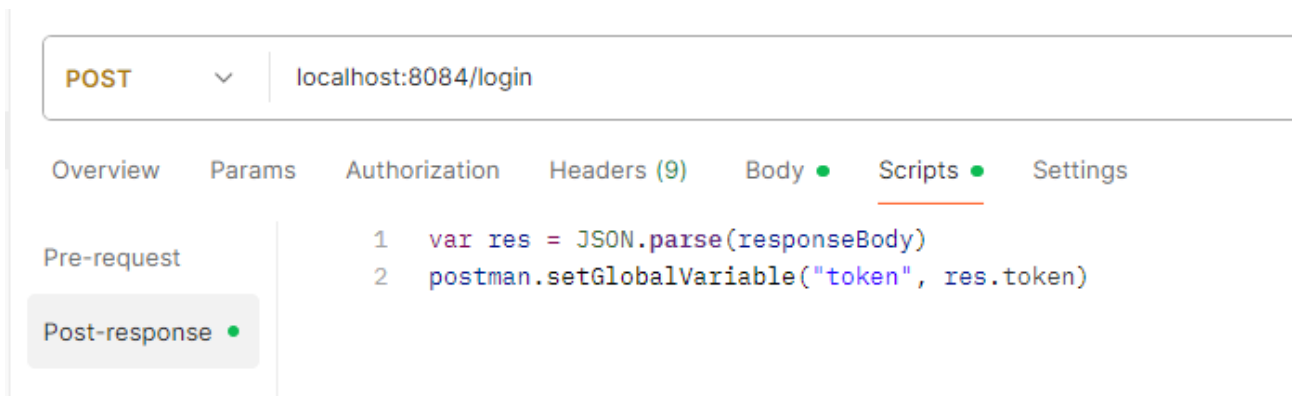
- Para probar con Postman se puede almacenar el token una vez generado con el siguiente código en *scripts*:

```
var res = JSON.parse(responseBody)
postman.setGlobalVariable("token", res.token)
```

Usuario y contraseña



Guardado del token para futuras peticiones



Uso del token guardado en otra petición

